Checkpoint Replay Support for Chakra and ET-Replay

Yantian yin

1. Introduction

This project aims to expand the functionality of Chakra and ET-Replay, benchmarking tools used to analyze and simulate AI training workflows. Currently, Chakra lacks support for replaying checkpointing behavior — a critical component in modern deep learning pipelines, where model states are periodically saved for fault tolerance and recovery. To address this limitation, the project introduces checkpoint replay functionality through three main stages:

- **Metadata Capture:** Identify and store all necessary metadata required to represent checkpointing events during model training.
- **Trace Integration:** Link device and host traces while injecting the captured checkpoint metadata to generate a trace that includes checkpoint operations.
- **Simulation:** Extend ET-Replay to simulate checkpointing behavior based on the enriched trace, allowing realistic benchmarking and performance analysis.

2. Brief Introduction to Chakra

Chakra is the benchmarking tool used in this project. It captures execution data from AI training workloads through the Kineto profiler and converts the collected information into a standardized Chakra trace format. This trace is then used to simulate execution timing and performance characteristics.
The typical benchmarking workflow in Chakra consists of the following steps:

1.Profiling Execution
The training job is executed with the profiler enabled. This produces:

- A host trace (CPU operations) in Chakra format — contains operation metadata but lacks precise timing information.
- A device trace (GPU operations) in Kineto format — includes detailed timing data.

2.Trace Linking
The host and device traces are merged, associating CPU operations with accurate timing and other execution context.

3.Simulation
The linked trace is fed into the Chakra simulator (et_replay) to analyze performance and generate benchmark timing results.

3. Training Model Setup

To generate profiling traces for benchmarking, we use a distributed training setup based on ResNet-50 with the CIFAR-10 dataset. The training is executed using PyTorch Distributed Data Parallel (DDP) across two GPU workers to simulate a realistic multi-device workload.

Key configuration details:

Component	Description
Model	ResNet-50 pretrained on ImageNet
Dataset	CIFAR-10, resized to 224×224
Batch Size	32 per GPU
Number of Workers	2 (DDP)
Optimizer	SGD (lr = 0.001, momentum = 0.9)
Loss Function	Cross-Entropy
Epochs	2
Checkpointing	Model + optimizer state saved every step

Checkpoint metadata details:

```
checkpoint = {
    'epoch': epoch,
    'step': step,
    'batch_size': batch_size,
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'loss': loss.item(),
    'rank': rank
}
with record_function("CHECKPOINT_Save"):
     torch.save(
        checkpoint,
        f'./checkpoints/checkpoint_step_{step}_worker_{rank}.pth'
    )
```

4. Gathering checkpoint metadata

To record the information required for later checkpoint simulation, we intercept every checkpoint save operation by hooking *torch.save( )*. This functionality is implemented in a new module: *checkpoint_hook.py*
We specifically extract the checkpoint file size for later simulation.

```
_checkpoint_metadata = {}
_metadata_file_path = None

# store the metadata in the global checkpoint metadata dictionary
function torch_save_with_hook(obj, f, *args, **kwargs):
    checkpoint_path = f
    if obj is dict and checkpoint_path:
        metadata = extract(obj, ['step', 'epoch', 'rank', 'batch_size'])
        metadata['checkpoint_path'] = checkpoint_path
        with lock:
            _checkpoint_metadata[checkpoint_path].append(metadata)
    return original_torch_save(obj, f, *args, **kwargs)

# flush the checkpoint_metadata to the provided path, will need this new file for the
linking step
function flush_checkpoint_metadata(rank):
    with lock:
        for path, entries in _checkpoint_metadata.items():
            total_bytes = _compute_total_storage_bytes(path)
            for entry in entries:
                entry['total_storage_bytes'] = total_bytes
        write_json(_checkpoint_metadata, _metadata_file_path)
```

This will give us new files that contain the metadata of each checkpoint

5. Injecting Checkpoint Nodes into the Execution Trace

To use checkpoint metadata and embed it into the execution trace, we first examined how existing operations are represented within Chakra's trace format. Specifically, we reviewed et_def.proto, which defines the data structures used to encode execution trace information.

In Chakra, the execution trace is composed of a set of nodes, where each node corresponds to a single operation executed during the training workflow. Each node generally includes the following fields:

- ID
- Name
- ***Type***

- Dependencies - Control dependencies and Data dependencies
- Timing information - start time and duration in microseconds
- Input/Output Metadata
- ***Attributes*** - additional key-value metadata

To incorporate checkpointing into the trace, we introduced a new node type: CHECKPOINT_NODE. This enables us to represent each checkpoint save operation as a dedicated trace node. Once this node type is registered in the trace schema, the next challenge is actually generating and inserting these nodes into the final Chakra trace.

This requires two major steps:

Step 1 — Linking Host and Device Traces with Checkpoint Metadata

During profiling, the CPU trace and GPU trace are generated separately. However, the checkpoint metadata is recorded only on the host side through our custom hooks, the trace linker must read the exported checkpoint metadata file and insert the corresponding CHECKPOINT_NODE entries at the correct execution positions.

To enable this, we extend the trace linker with an additional optional parser argument:

```
parser.add_argument(
        "--checkpoint-metadata",
        type=str,
        required=False,
        help="Path to checkpoint metadata JSON produced by checkpoint_hook",
    )
```

This argument is then passed into the main linking function:

```
linker.link(
        args.rank,
        args.chakra_host_trace,
        args.chakra_device_trace,
        args.output_file,
        checkpoint_metadata_path=args.checkpoint_metadata,
        )
```

With this input, the trace linker can locate the checkpoint metadata and begin inserting corresponding checkpoint nodes.

Checkpoint Node Insertion Logic

In the trace_linker, we append them by their step info stored in the control dependence after linking.

```
def link(
        self,
        rank: int,
```

```
        chakra_host_trace: str,
        chakra_device_trace: str,
        output_file: str,
        checkpoint_metadata_path: Optional[str] = None,
    ) -> None:
        ……
        function find_profiler_step_index(node):
          walk up node.ctrl_deps until find a "ProfilerStep#k" node
          return k (or None)

        flat_entries = all (ckpt_path, meta) where:
            meta.rank == current rank

        checkpoint_nodes = all nodes with name == "CHECKPOINT_Save"
        for each such node:
          compute profiler_step_index(node)
        pair checkpoint_nodes with flat_entries
        for each pair:
          attach attrs: path, epoch, step, rank, batch_size, total_storage_bytes
```

Step 2 — Converting Checkpoint Operations into CHECKPOINT_NODE

After inserting checkpoint operations into the linked trace, the next step is for the converter (*pytorch_converter.py*) to correctly translate these operations from JSON format into their Chakra protobuf representation. To achieve this, we extend the converter so that any node whose name contains "CHECKPOINT" is mapped to the new protobuf enum: **CHECKPOINT_NODE**.

converter.py

```
def is_checkpoint_op(self) -> bool:
        return ("CHECKPOINT" in self.name)
```

And also in *pytorch_converter.py* we add this mapping to chakra protobuf format

```
def get_protobuf_node_type_from_json_node(
        self, json_node_map: Dict[int, PyTorchNode], json_node: PyTorchNode
    ) -> int:
        ……
        if json_node.is_checkpoint_op():
            return CHECKPOINT_NODE
```

This type mapping is then used inside the main conversion function:

```
def convert_json_to_protobuf_node(
        self,
```

```
        json_node_map: Dict[int, PyTorchNode],
        protobuf_node_map: Dict[int, ChakraNode],
        json_node: PyTorchNode,
        ) -> ChakraNode:
        ……
        protobuf_node = ChakraNode()
        protobuf_node.id = json_node.id
        protobuf_node.name = json_node.name
        protobuf_node.type = self.get_protobuf_node_type_from_json_node(json_node_map,
json_node)
        # pseudocode below
        for each attr_name in checkpoint_attrs:
              protobuf_node.attr.append(attr_name, dtype)
```

6. Simulation

we extend ET-Replay to support the execution of CHECKPOINT_NODE operations. Since the current ET-Replay implementation has no built-in behavior for checkpoint nodes, we introduce a simulation strategy that mimics the real checkpoint save process.

For each CHECKPOINT_NODE, ET-Replay performs the following:

1. Locates the checkpoint size from node attributes
2. Constructs a dummy checkpoint payload to approximate serialization work
3. Serializes and writes the payload to a user-specified output directory

We update the DFS traversal to detect checkpoint nodes and include them in the execution schedule

```
def dfs_traverse(node):
    # Check if node is a checkpoint
    if (
        node.type == NodeType.LABEL
        and node.name
        and node.name.upper().startswith("CHECKPOINT")
    ):
        self.sorted_nodes.append(node)  # Add to execution list
        checkpoint_nodes.append(node)    # Track as checkpoint
        ……
```

Placing checkpoint nodes in self.sorted_nodes ensures they are executed during replay:

```
def run_ops(event_1, event_2, iter):
    # ...
```

```
    for cnt, node in enumerate(self.sorted_nodes):
        success, _ = self.run_op(node, iter, cnt)
        if not success:
            return False
```

Checkpoint nodes are detected inside analyze_node()

```
def analayze_node(node):
            ……
            if (
                node.type == NodeType.LABEL
                and node.name
                and node.name.upper().startswith("CHECKPOINT")
            ):
                return True, ""
```

By modifying the above function we make sure that checkpoint node is in the sorted_nodes

```
picked_nodes = []
        for node in self.sorted_nodes:
            success, msg = analayze_node(node)
            if success:
                picked_nodes.append(node)
            else:
                self.add_skipped_nodes(node, msg)
        self.sorted_nodes = picked_nodes
```

Then for a checkpoint node we read the size of the node and create dummy checkpoint dictionary by inserting random floats base on that info, serialize them like what pytorch.save did and write it to a path user provided

```
dummy_checkpoint = {
    'epoch': 0,
    'step': 0,
    'batch_size': 32,
    'model_state_dict': {},
    'optimizer_state_dict': {},
    'loss': 0.0,
    'rank': 0
}

# here we make assumption that the checkpoint size is split evenly between model and
optimizer state
target_data_size = int(checkpoint_size)
model_state_target = checkpoint_size // 2
optimizer_state_target = checkpoint_size // 2
```

```
#assume 1mb for each array
bytes_per_array_raw = 1024 * 1024
bytes_per_array_serialized = int(bytes_per_array_raw)

# Calculate number of arrays needed for each dict
num_model_arrays = max(1, model_state_target // bytes_per_array_serialized)
num_optimizer_arrays = max(1, optimizer_state_target // bytes_per_array_serialized)

# Add dummy numpy arrays to model_state_dict
for i in range(num_model_arrays):
   array_size = bytes_per_array_raw // 4  # FP32 = 4 bytes per element
   dummy_checkpoint['model_state_dict'][f'layer_{i}.weight'] =
np.random.randn(array_size).astype(np.float32)


# Add dummy numpy arrays to optimizer_state_dict
# Optimizer state typically has similar structure (momentum buffers)
for i in range(num_optimizer_arrays):
   array_size = bytes_per_array_raw // 4  # FP32 = 4 bytes per element
   dummy_checkpoint['optimizer_state_dict'][f'layer_{i}.weight'] = {
   'momentum_buffer': np.random.randn(array_size).astype(np.float32)
   }


# Actually serialize it
start_serialize = time.time()
serialized = pickle.dumps(dummy_checkpoint, protocol=pickle.HIGHEST_PROTOCOL)
serialize_time = time.time() - start_serialize

# Write the serialized data
with open(output_path, "wb") as f:
   f.write(serialized)
```

7. Time Evaluation

To validate the accuracy of checkpoint replay, we compared the measured checkpoint durations during real training with the simulated durations generated by ET-Replay.

For actually training :

Rank	Epoch	Step	Checkpoint Time (ms)	Iteration Time (ms)
0	0	0	2577.87	6782.92
1	0	0	4252.18	8170.17
1	0	1	2921.17	3044.96
0	0	1	3673.59	5583.72
0	1	0	3765.32	3888.93
1	1	1	3520.08	4107.51
0	1	1	3782.60	3897.41
1	1	0	3293.83	3617.67
0 AVG			3449.84	
1 AVG			3496.82	
All AVG			3473.33	

Simulation :

```
[2025-12-12 04:45:18,004] et_replay.py:1508 [INFO]: iteration = 14
[2025-12-12 04:45:21,525] et_replay.py:1209 [INFO]: Simulated checkpoint save:
./checkpoints/checkpoint_step_0_worker_0.pth (204835490 bytes, serialization:
129.64ms)
[2025-12-12 04:45:22,888] utils.py:44 [WARNING]: Chrome profile trace written to
/tmp/tmp_20251212_6f64d26_3172238.json
[2025-12-12 04:45:23,001] et_replay.py:1628 [INFO]: Replay finished
[2025-12-12 04:45:23,002] et_replay.py:1629 [INFO]: Replay time per iteration:
3435.0263671875 ms
[2025-12-12 04:45:23,002] et_replay.py:1632 [INFO]: Checkpoint avg time: 3295.15 ms
over 10 checkpoints
[2025-12-12 04:45:23,002] et_replay.py:1635 [INFO]: Operator coverage: 1051 / 1054 =
0.9971537001897534
[2025-12-12 04:45:23,003] et_replay.py:1643 [INFO]: FB internals not present
[2025-12-12 04:45:23,004] et_replay.py:1814 [INFO]: Replay finished successfully.
```

Time difference is below 10%

8. Conclusion

This project successfully extends Chakra and ET-Replay with full support for checkpoint replay. By introducing checkpoint metadata capture, integrating checkpoint operations into the trace linking pipeline, and implementing a detailed checkpoint simulation mechanism inside ET-Replay, the system can now model checkpoint behavior with high fidelity.

The solution captures real checkpoint characteristics—including file size, current epoch and step, serialization cost—and reproduces them during replay using synthetic checkpoint payloads. Experimental evaluation shows that the simulated checkpoint durations differ from real training measurements by less than **10%**, demonstrating that the approach provides accurate performance modeling without requiring access to the original training run.

9. Video of the full run

https://youtu.be/IR1J9kq53H0

10. Source code

https://github.com/yyt0823/chakra_replay_checkpoint

References

[1] **MLCommons Chakra** — *A Trace-Based Performance Model for ML Training Workloads*.
Available at: https://github.com/mlcommons/chakra

[2] **ET-Replay (Chakra Simulator)** — *Execution Time Replay Engine for Chakra Traces*.
Available at: https://github.com/mlcommons/chakra/tree/main/et_replay

[3] **Kineto Profiler** — *PyTorch Profiler Back-End for CPU/GPU Activity Collection*.
Available at: https://github.com/pytorch/kineto

[4] **PyTorch Distributed Data Parallel (DDP)** — *High-Performance Multi-GPU Training API*.
Available at: https://pytorch.org/docs/stable/generated/torch.nn.parallel.DistributedDataParallel.html

[5] **PyTorch Checkpointing** — *torch.save / torch.load Documentation*.
Available at: https://pytorch.org/docs/stable/generated/torch.save.html

[6] **CIFAR-10 Dataset** — *Canadian Institute for Advanced Research (CIFAR)*.
Available at: https://www.cs.toronto.edu/~kriz/cifar.html

[7] **ResNet-50 Architecture** — He, K., Zhang, X., Ren, S., & Sun, J. *Deep Residual Learning for Image Recognition.* CVPR 2016.
Available at: https://arxiv.org/abs/1512.03385